

ZYVOR · SHUKRA

See inside every VM, **without going inside it.**

A brochure for teams that run KVM: how Shukra watches every QEMU workload from the hypervisor with eBPF, what happens step by step when a VM turns slow or its traffic stops, how it watches the VMM itself, and how a VM can be held to a policy or cut off under guardrails.

OBSERVE

KVM exits, run-queue delay, disk latency, every TCP connect a guest makes and the names it asks for. No agent in the VM.

EXPLAIN

Ranked host-side causes with the evidence, for now or for 03:12 last night.

PROTECT

Detection rules, learned baselines, VMM tripwires, an egress policy tried in audit first, and isolation of a VM without touching it.

SAY WHAT IT CANNOT SEE

Every answer lists its blind spots. A program that is not measuring says so.

RECENTLY ADDED

- Egress policy
- VMM tripwires
- The daemon's own cost, cut
- TLS server names
- Guarded responses
- Learned baselines
- Right-sizing advice
- Noisy-neighbour map
- Explain a past time
- Incident bundles
- Who took the vCPU's CPU
- FluxVM guests
- DNS names the guest looks up
- Kernel drops and handshake outcomes
- A share of events for each kind
- Filterable connection tables
- Tested on real KVM guests

Standalone eBPF for KVM hosts · QEMU, libvirt, kubevirt and FluxVM · x86_64 and arm64 packages · no tagged release yet

Open source under the Apache License 2.0. Copyright 2026 Zyvor.

SHUKRA

No agent inside the VM.

Root is the hypervisor. Identity comes from the host: the QEMU command line, or FluxVM's store. Host connections are the VMM's own; the guest's are seen on its tap when the tap program is attached.

3
virtual machines

77
programs attached

3
detections

observe
mode

BOUNDARY

What this build will not say

CPU steal as the guest counts it is not measured. The host's view of it, vCPU preemption, is on the Scheduler page.

In-guest process identity is not measured.

observe: host traces, and guest traffic on the VM taps (3 taps: tap1, tap2, tap7)

Illustrative: the Shukra overview, rendered from the console's fixture data. UI captures in this brochure are fixtures, not customer data; diagrams are conceptual.

When teams reach for Shukra

A guest can tell you it is slow. Only the host knows why: whether a neighbour took its CPU, the disk stalled, the host dropped its packets or the guest stopped reading its NIC. Shukra sits on the hypervisor and answers that, without installing anything in the VM.

Problems Shukra addresses

- 1 Guest metrics cannot see the host, so "is it us, the disk or a noisy neighbour?" is a guess.
- 2 Slowness is reported after the fact, and the counters that would explain it have already moved on.
- 3 A compromised VM has to be cut off now, without logging in to it and without cutting off the people managing it.
- 4 Agents inside guests are a fleet to install, patch and trust. Someone else's VM is not yours to change.
- 5 VMs are sized once and never revisited: some idle at eight vCPUs while others wait for a CPU.
- 6 The first unusual connection from a VM has no rule yet, because nobody knew to write one.
- 7 A QEMU process is where a guest escape lands, and on most hosts nothing notices when one opens `/etc/shadow` or calls `ptrace`.
- 8 Every tool should say what it cannot see. A missing measurement must never look like a healthy zero.

What Shukra is, in one sentence

Shukra is eBPF runtime intelligence and security for KVM: a daemon on the hypervisor that attaches kernel traces, joins them to the QEMU process, and gives an operator a console and a CLI that say what they know, how they know it, and what they cannot see.

Two programs you run

shukrad (on the hypervisor)

Privileged. Loads the eBPF programs, scans `/proc` for QEMU and FluxVM guests, keeps the flight recorder and detection rules, and serves the API and the console on port 30970.

shukractl (anywhere)

An operator CLI that talks to the API only and never loads BPF. Status, doctor, traces, explain, advise, baseline, policy, actions, incident, watch, isolate: one screen to trust first, then the detail.

At a glance

7

eBPF programs, each loaded on its own so one missing tracepoint detaches only that program

31

kernel attach points across kvm, sched, block, net, tap, drops and vmm

0

agents, modules or changes inside any guest

16

console pages in five groups: overview, investigate, diagnostics, network and security

37

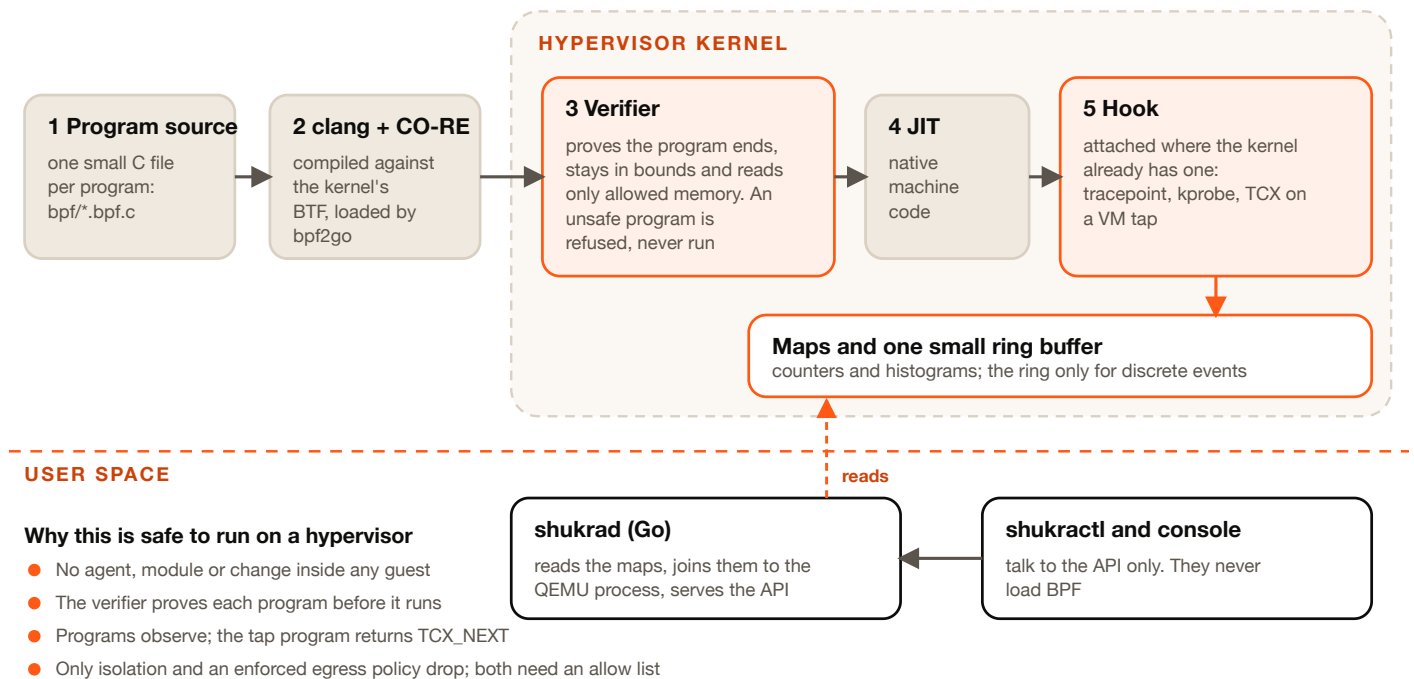
HTTP routes: three for operations and 34 under `/api/v1`, eight of which change state and need the admin key

21

shukractl commands, including `explain`, `advise`, `policy`, `approve`, `isolate` and `doctor`

How eBPF does it

eBPF lets Shukra run small, verified programs inside the Linux kernel at the points where VM exits, scheduling, disk requests and packets already pass. That is how it sees a guest from outside: no kernel module, no agent, no change to the VM.



From source to a kernel hook and back to the daemon. Illustrative.

What Shukra attaches to

Tracepoints for KVM exits, the scheduler, block, dropped packets and VMM syscalls; kprobes on TCP connect; and TCX on each VM's tap, on Linux 6.6 and newer.

How the data comes out

Hot paths stay in maps as counters and histograms. A small ring buffer carries only discrete events, and every source of events is sampled or rate-limited.

What keeps it safe

The verifier refuses anything unsafe, and each program is its own object. A kernel that cannot run one reports it detached, with the reason, and the others keep running.

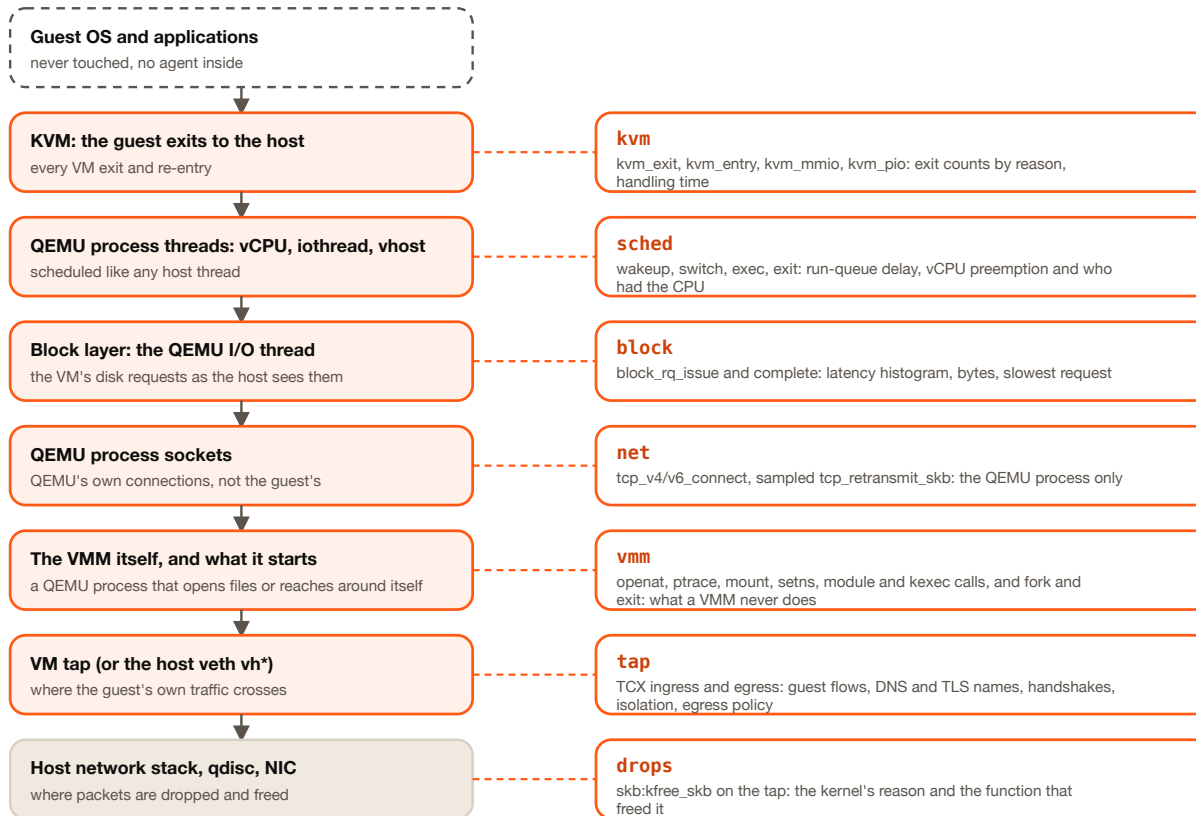
Built once, run on the kernel in front of it. The programs are compiled with CO-RE against kernel type information (BTF) and loaded by `bpf2go`, so one object adapts to the running kernel. Without BTF, clang or root, Shukra still discovers VMs, reports every program as detached with the reason, and never invents a counter.

Why eBPF, not the alternatives

	Agent inside each guest	Kernel module	Shukra with eBPF
Sees host-side causes	no: it only sees the guest's view	yes	yes: exits, run-queue delay, preemption, block, drops
Changes to guests	one install per VM, kept patched	none	none
Risk to the host	lives in someone else's VM	a bug can crash the hypervisor	the verifier proves each program safe before it runs

Where Shukra sits on the hypervisor

A VM's life crosses the same layers on every host. Shukra places one program at each layer that matters and reads names and threads from `/proc`, so the picture of a slow or silent VM runs from the guest's exits to the last function that freed its packet.



VM names, threads and tap names come from `/proc` and the QEMU command line, not from BPF or the guest.

Why the KVM and scheduler hooks

A VM is slow for host-side reasons first: it exits too often, or its vCPU threads are runnable and waiting for a CPU. Both are visible only from the host.

Why the VMM hooks

A QEMU process in steady state opens no files and makes none of the calls a normal program makes to reach around itself. When one does, a guest may have escaped into it.

Why the tap

The guest's own traffic crosses the host side of its tap. Seeing it there gives the guest's addresses and the names it asks for, with no agent and without reading application data.

Why the drop tracepoint

Most packet loss is decided inside the host kernel. The tracepoint says why, and which function freed the packet, so a firewall rule and a full queue are told apart.

The seven eBPF programs

Seven programs, 31 attach points (4 + 4 + 2 + 3 + 2 + 1 + 15). Each is a separate object with its own hooks. A host that cannot run one loses only that program, and says why.

Program	Hooks	What it records	Needs
kvm	<code>kvm_exit</code> , <code>kvm_entry</code> , <code>kvm_mmio</code> , <code>kvm_pio</code>	exit counts by reason, exit handling time as a histogram (halts excluded) and time per reason	BTF, Linux 5.8+
sched	wakeup, switch, exec, exit	on-CPU time, run-queue delay per thread, vCPU preemption (runnable but off a host CPU, and who had it), exec and exit of QEMU children	BTF, Linux 5.8+
block	<code>block_rq_issue</code> , <code>block_rq_complete</code>	latency histogram, requests, bytes and the slowest request, per direction	BTF, Linux 5.8+
net	<code>tcp_v4_connect</code> , <code>tcp_v6_connect</code> , sampled <code>tcp_retransmit_skb</code>	exact connect counts (IPv4 and IPv6) and 1-in-64 retransmit samples: the QEMU process's sockets	BTF, Linux 5.8+
tap	TCX ingress and egress on each VM tap	the guest's own traffic: per-tap counters; an event per TCP connect, new UDP flow and connection made to the guest; the name in each DNS query and TLS ClientHello; each handshake's outcome; isolation and egress policy	Linux 6.6+, <code>CAP_NET_ADMIN</code>
vmm	13 syscall tracepoints (three opens, ten calls a VMM never makes), plus process fork and exit	for a QEMU process and what it started: each file it opens and each sensitive call it makes (<code>ptrace</code> , <code>mount</code> , <code>setns</code> , module loads). A steady-state VMM does none	BTF, Linux 5.8+
drops	<code>skb:kfree_skb</code> on each VM tap	what the kernel dropped on the tap, by its own reason and the function that freed it, with Shukra's own isolation subtracted	Linux 5.17+

Never in the way of other tools

The tap program returns `TCX_NEXT`, never `TCX_PASS`, so any other program on the same tap (Cilium, FluxVM's egress program, a `tc` filter) still runs after it. Its links and maps are pinned under `/sys/fs/bpf/shukra/tap`.

Kernel baselines

Any program needs BTF at `/sys/kernel/btf/vmlinux` and `CAP_BPF` (Linux 5.8+). Drop reasons need 5.17; TCX, egress policy and isolation need 6.6. On arm64, `kvm_pio` does not exist, so `kvm` attaches 3 of its 4 hooks.

Identity comes from the host

Shukra names a VM from the QEMU command line (`-name` or `guest=`, `-uuid`, `ifname=`) and, for libvirt taps passed as file descriptors, the process's `fdinfo`. A FluxVM guest (QEMU, Cloud Hypervisor, Firecracker or `fluxvm-hypervisor`) is named from FluxVM's own record and traced on its host veth. A PID that is not a VMM rolls up to `_host` as `unattributed`, never a made-up VM name.

What a host reports when a program cannot run

Host	Program	Reported
hv-2	tap	detached, with the reason: the kernel is older than 6.6
hv-3	vmm	detached: turned off with <code>-vmm-tripwires=false</code>

Illustrative. `shukractl programs` shows this per host, and a VM with no measurement has no series at all: "not measured" is never rendered as zero. Percentiles come from log2 buckets and can read up to 2x high.

What it costs, measured

A program that runs on every packet or every open has to be cheap, and has to say what it costs. Each figure below was measured by the project and is quoted from its own documentation. Scale it by your own kernel, hardware and traffic.

What	Measured cost	Where, and the caveat
drops	about 255 ns per drop; 69 a second is 0.002% of one CPU	12-CPU Xeon hypervisor. It runs once per packet dropped anywhere on the host, so scale by your drop rate: a million a second is about a quarter of a CPU
sched	about 490 ns per switch with no VM watched, about 615 ns with ten; about 435 ns before preemption tracking	same host, ten VMs, about 60,000 switches a second: about 11 ms of CPU per second more, under 0.1% of the host. Includes the kernel's own run-time accounting, switched on for the measurement
vmm	about 183 ns per open; 0.161% of one core at about 8,100 tracepoint hits a second; about 2% of a core at 100,000 opens a second	Intel Xeon E-2336, Linux 6.8, a k3s and Cilium node. The tracepoints fire for every process, so each first finds out whether the caller is a VMM. <code>-vmm-tripwires=false</code> on a host that busy
tap: TLS server names	about 10 to 15 ns added to a data segment; nothing to an ACK or a UDP datagram; about 60 to 85 ns for the one segment per connection that is a hello	same Xeon, on a private copy of the program. A floor: the same packet run over and over, caches warm. About 1 to 1.5% of a core at a million data segments a second
tap: egress policy	with no policy on a tap, one extra read and compare per packet: about 7 to 10 ns (a data segment 76 to 78 ns before, 86 to 87 ns after)	about 1% of a core at a million packets a second. A tap under a policy adds a trie lookup per new connection or datagram
the daemon	5 to 9% of a core and about 88 MB, down from about 55% and 155 MB, with the same output	found by profiling <code>shukrad</code> on a 12-core production node: 10 VMs, k3s and Cilium. Nearly all the old cost was garbage collection

Measured on lab and production hypervisors of the project's own. Costs of `kvm`, `block` and `net` are not published.

How the daemon went from about 55% of a core to 5 to 9%

The flight recorder is a real ring

Once a VM's ring was full, every event copied all 4,096 events into a new slice: over a megabyte of pointers for the garbage collector, per event. Adding to a full ring now allocates nothing, and a test asserts it.

Thread counters are summed as read

The scheduler maps hold every thread that has run on the host, about 64,000 on that node. Each 2-second refresh built a record for each only to add it into the host row. They are now added into one slot as they are read.

Large maps are read in batches

One system call for up to 2,048 entries instead of two per entry: over 200,000 system calls per refresh on that node. A kernel or map that cannot batch falls back to the old walk.

Same output, and tests that say so. A test feeds the same readings through the old and new paths and checks that the scheduler rows and per-thread rows the API serves are identical.

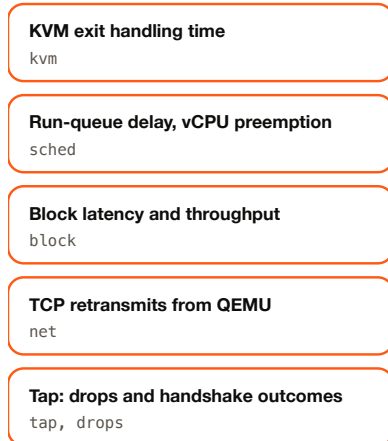
Measure your own host. `scripts/bench-tap.sh` reports what the tap program costs per packet on your kernel. It runs `BPF_PROG_TEST_RUN` on a private copy with its own maps, attached to nothing, so it is safe on a production host.

SLOW-VM SCENARIO · PART 1 OF 4

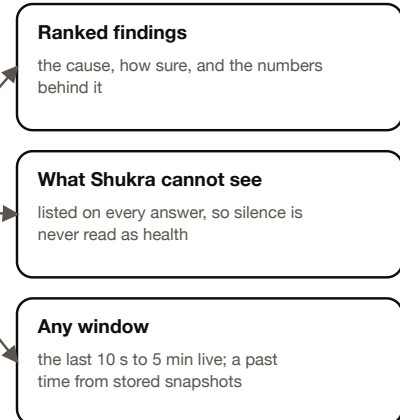
A VM turns slow

It is 03:12. A user says **payment-prod-03** is slow. Nothing inside the guest looks wrong, and the host graphs are flat. The counters that would explain it are already in Shukra's maps, so the operator asks.

EVIDENCE THE HOST ALREADY HAS



WHAT THE OPERATOR GETS



- 1 **The programs count, always.** Exits, preemption, disk requests, connects and tap drops are recorded before anyone asks.
- 2 **shukrad joins them to the VM.** It rescans `/proc` every two seconds, so counters land on the right process and thread: a vCPU, an iothread or a vhost worker.
- 3 **The operator runs `shukractl explain payment-prod-03`** (or opens the Explain page). It weighs the last minute; `--window` takes 10 s to 5 min.

Nine host-side causes, best supported first

Cause	What it points at
host_cpu_contention	the VM's threads wait for a host CPU after being woken: host load, pinning, neighbours
cpu_preempted	the host took CPU away from the vCPUs while they wanted to run
noisy_neighbour	one other VM accounts for at least half of that preempted time
storage_latency	block requests from QEMU take long: the backing device, queue depth, other writers
kvm_exit_handling	the host is slow to service KVM exits, often device emulation
tcp_retransmits	the QEMU process retransmits: host traffic such as migration or a remote disk, not the guest's
guest_traffic_dropped	the kernel drops this VM's packets and Shukra's isolation is not the cause: another program on the tap
guest_not_reading_nic	the tap's queue is full: the guest is stalled, has no working NIC driver or is overloaded
guest_connects_failing	most outbound connections fail: refused means nothing listens, never answered means something drops the SYN

A finding says where to look, not what is wrong inside the guest. Each carries a cause, a confidence (high, medium or low), a summary and the evidence behind it. When nothing crossed a threshold the answer is `no_host_cause` : look inside the guest, which Shukra does not measure.

SLOW-VM SCENARIO · PART 2 OF 4

Who took my vCPU's CPU?

The finding is `cpu_preempted`: the host took CPU away from the VM's vCPUs. On a busy hypervisor the next question is who. Shukra answers it from the scheduler's own switch events, without touching either guest.

payment-prod-03: vCPU threads runnable but off a host CPU, 240 ms in the window

vm:batch-etl-01 190 ms 79%

kworker 50 ms

Who had the CPU

the thread that took it when a vCPU was switched out while still runnable: a VM by name, or a host command

One VM accounts for half or more

then Explain names it: cause noisy_neighbour. `cpu_preempted` stays beside it

What it was doing meanwhile

the culprit's own CPU, block and exit numbers over the same window

Illustrative numbers. The percentages are shares of the VM's preempted time.

How it is measured

`sched_switch` fires when a thread leaves a CPU. If a QEMU thread leaves while still **runnable**, it was taken off while it wanted to run: the program remembers who took the CPU and, when the thread next runs, adds the wait to the pair (this thread, that thread). A thread that goes to sleep is dequeued first, so sleeping is never counted.

What the answer names

- Another VM, as `vm:<name>`, or the VM's own other threads.
- Or a host command with its instance suffix removed: `kworker/3:1` and `kworker/u16:2` are both `kworker`.
- For each culprit VM, what it was doing over the same window: its on-CPU time and KVM exits.

What it is not

- Not the steal counter a guest reads. It is the host's view of the guest losing its processor.
- vCPU threads only. An iothread that waited for a CPU is not the guest losing its processor.

```
$ shukractl trace contention --window 5m
vm=osboxes-debian preempted=1882612ns over 12 preemptions (other VMs 0ns, its own threads 0ns, host tasks 1882612ns)
taken by host tasks: cilium-agent 1031780ns, kubectrl 321523ns, hubble-relay 188161ns, containerd-shim 103092ns
as printed in the after-the-fact tutorial: here the culprit was the host's own tasks, not another VM
```

CONTENTION

Whose CPU another VM took (last 1m0s)

VICTIM	TAKEN BY VM	TIME	SHARE OF ITS PREEMPTION
payment-prod-03	api-01	190 ms	79%
redis-01	api-01	60 ms	63%
api-01	payment-prod-03	25 ms	36%

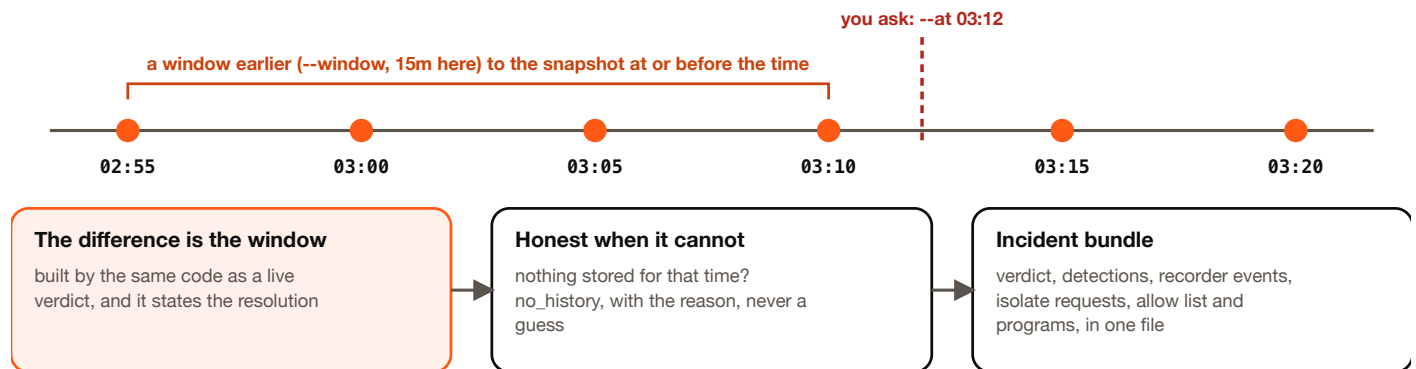
Illustrative: the Contention page, rendered from fixture data.

SLOW-VM SCENARIO · PART 3 OF 4

Why was it slow at 03:12?

It is half past five. The last minute, which Explain weighs by default, is no help. With a data directory, the daemon has been storing a coarse snapshot every five minutes, so the operator can ask about a past time and say how far to trust the answer.

A COARSE SNAPSHOT EVERY 5 MINUTES (with `-data-dir`)



- 1 Ask for the verdict at that time.** `shukractl explain payment-prod-03 --at -2h30m`, or an RFC 3339 time. The window is 5 minutes to 6 hours, 15 minutes by default.
- 2 Read the resolution line first.** Snapshots are five minutes apart, so this is an average around the time, not the second it happened. A short burst can be diluted into nothing, and the answer says so.
- 3 Find who was involved** with `shukractl trace contention` if the finding is `cpu_preempted` or `noisy_neighbour`.
- 4 Attach one file to the ticket:** `shukractl incident payment-prod-03 --at -2h30m --out bundle.json`, written with mode 0600.

```
$ shukractl explain osboxes-mint --at -2h30m
EXPLAIN why was this VM slow? (window: 5m0s) (at: 2026-09-20T04:50:27Z)
  resolution: 5m0s snapshots: the verdict stands on 2026-09-20T04:40:53Z and 2026-09-20T04:45:53Z
  findings (best supported first)
    [low] no_host_cause: Nothing host-side crossed a threshold. If the guest is slow, the cause may be inside it, which
    this build does not measure.
as printed in the after-the-fact tutorial. A calm host is a real answer: it says the host was fine.
```

What the incident bundle holds

The verdict; the window's detections; what the flight recorder saw of that VM (at most 500 events); isolate and release requests; whether isolation is available and the management networks an isolated VM keeps; and which programs were attached. Nothing else from the daemon's configuration: no key, no listen address.

What it cannot tell you

The second it happened. Anything before the daemon first stored a snapshot, or while it was not running. Anything inside the guest. And with nothing stored, it says `no_history` and why, instead of guessing. Retention is set by size: about a day or two at ten VMs.

The bundle is as sensitive as the event list. It names your VMs, the addresses they connected to and the DNS names they looked up. Treat it like the event list.

SLOW-VM SCENARIO · PART 4 OF 4

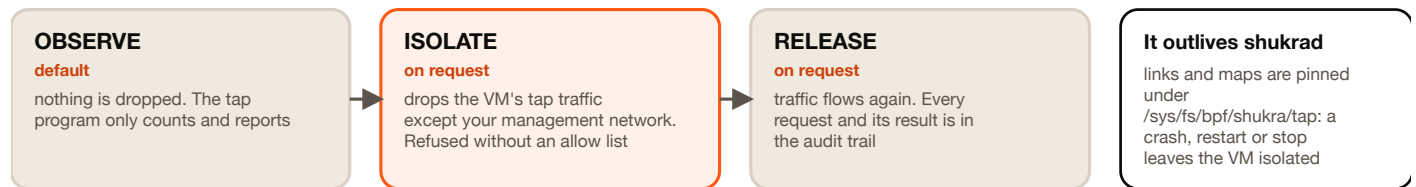
Lost traffic, then contain

Another ticket: **api-01** cannot reach a service. Is the destination refusing, is something dropping the packets, is the guest not reading its NIC, or is Shukra itself in the way? The tap answers each connection, and the kernel names each drop.

EVERY TCP CONNECT THE GUEST MAKES ENDS AS ONE OF FOUR



CUT A VM OFF WITHOUT TOUCHING IT



What the kernel dropped, and why

Reason on the tap	What it means	Where to look
TC_INGRESS, TC_EGRESS	a tc or TCX program on the tap dropped a frame; if Shukra accounts for it, that was isolation	<code>bpftool net show dev <tap></code> : another program (Cilium, a tc filter)
FULL_RING	the tap's queue is full: the guest is stalled, has no working NIC driver, or is overloaded	the VM itself
NETFILTER_DROP	iptables or nftables	the rule whose packet counter is moving
QDISC_DROP, CPU_BACKLOG	the host's own queues are overflowing	host load and qdisc settings

Each drop is also attributed to the function that freed the packet. Shukra's own isolation drops are subtracted, so "is it us?" has an answer: `shukractl trace drops`. Reason names are read from the running kernel's BTF and never hard-coded; one it cannot name is shown as `reason N`, not guessed.

- 1 Read the outcome.** `shukractl trace tap --vm api-01` shows accepted, refused, never answered and blocked per direction, with the guest's handshake time.
- 2 Read the drops.** `shukractl trace drops --vm api-01` says what the host kernel dropped on the tap, by reason and function, and whether it was Shukra.
- 3 If the VM must be cut off,** `shukractl isolate api-01` drops its tap traffic except your management network. It prints whether the kernel took the change.

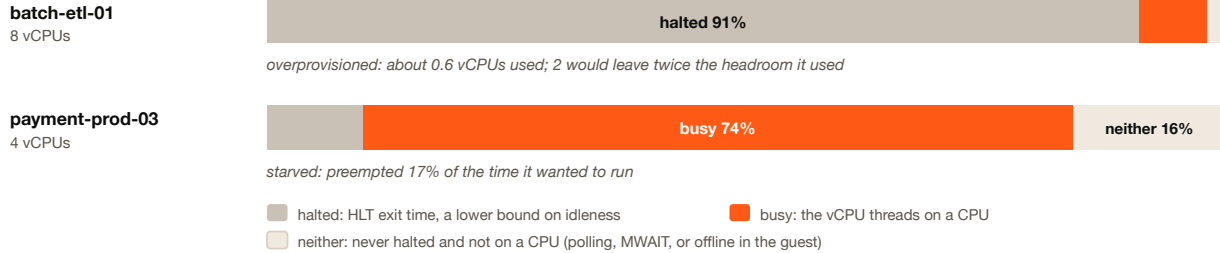
Isolation is real, and conditional. It needs an explicit allow list (`-isolate-allow`) and Linux 6.6 or newer, and is refused without them, because a VM that could reach nothing could not be reached by the host that isolated it. `applied` is true only after the kernel took the change on every tap; a half-applied isolation is reported `partial`, not silently reopened. It blocks the VM's tap: it does not touch vhost-user or SR-IOV interfaces.

BEYOND THE INCIDENT · CAPACITY

Is this VM the right size?

A VM that idles at eight vCPUs wastes host CPU, and one that wants CPU and is not getting it costs its users. The counters Shukra already has answer both, in both directions. The answer is advice for a person, with the numbers and how sure it is. Nothing here resizes a VM.

WHAT THE ADVICE STANDS ON: THE SHARE OF EACH VM'S vCPU CAPACITY OVER THE WINDOW



Overprovisioned

2 or more vCPUs, halted at least 80% and busy at most 20%

Starved

busy at least 50%, and preempted 10% or a vCPU waiting 2 ms for a CPU

Nearly idle

one vCPU, halted at least 95%: a consolidation candidate

Illustrative fixture numbers, over a five-minute window.

What it prints

`shukractl advise` shows, per VM, the share of the window its vCPUs were halted, busy and preempted, then advice: `overprovisioned` (with a size that leaves twice the headroom it used), `starved`, `nearly_idle`, `no_change`, `not_enough_data` or `idle_unavailable`.

A lower bound, and a gap it shows

Idleness is HLT exit time, so a guest that idles by polling reads as busy. The share that is neither halted nor on a CPU is reported, and when it is 30% or more, `no_change` says that is not proof the VM is right-sized.

Only where the CPU names the exit

Exit reasons are named on Intel hosts. Elsewhere Shukra says `idle_unavailable` and claims nothing about over-provisioning; starvation is still reported.

How sure

Every piece of advice carries a confidence and its evidence. A window under three minutes lowers it; under 20 seconds of history the answer is `not_enough_data`. Only vCPU threads count.

RIGHT-SIZE							
Is each VM the right size? (last 5m0s)							
VM	VCPUS	HALTED	BUSY	PREEMPTED	NEITHER	WORST RUN-QUEUE WAIT	ADVICE
batch-etl-01	8	91%	7% (0.6 vCPUs)	0%	2%	65.5 µs	overprovisioned MEDIUM This VM has 8 vCPUs and used about 0.6 of them. 2 would leave it twice the headroom it used. Halted 91% of the time.
payment-prod-03	4	10%	74% (3.0 vCPUs)	17%	16%	33 ms	starved HIGH This VM wants more CPU than the host gives it. Preempted 17% of the time it wanted to run.

Illustrative: the Right-size page, rendered from fixture data. It sits in the Diagnostics group.

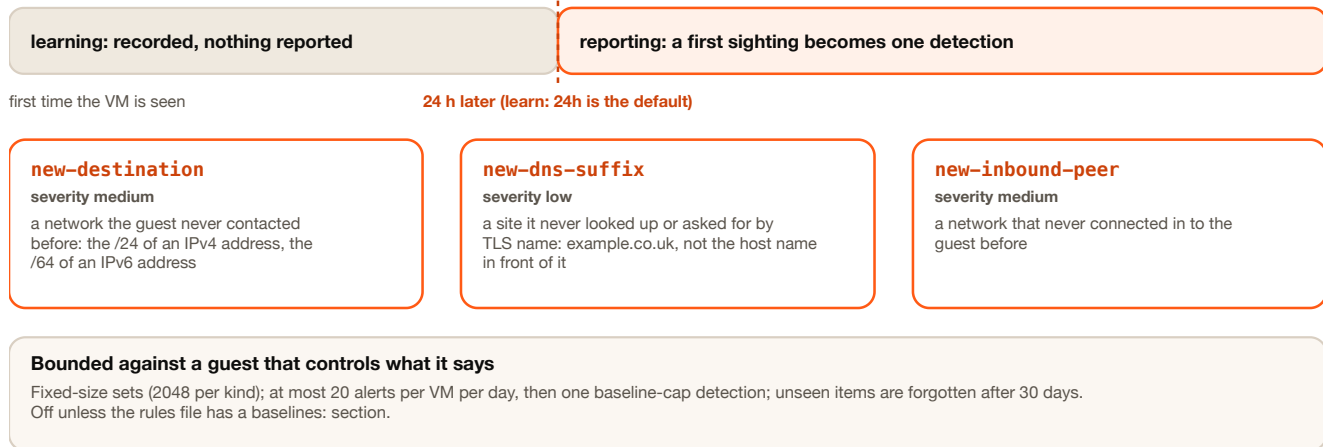
Advice, not an action. Starved VMs are told to reduce what they compete with (pinning, moving a neighbour) before adding vCPUs, because more vCPUs mean more threads competing for the same host CPUs. A person decides.

BEYOND THE INCIDENT · WHAT IS NEW

What is new for this VM?

A watchlist needs someone to know what is bad. A baseline needs no one to know: it learns what each VM normally does and reports the first time it does something new. A web server that has talked to the same three networks for a month and suddenly talks to a fourth is worth a look, and no rule had to name that network in advance.

A VM IS ONLY OBSERVED FIRST; THEN THE FIRST SIGHTING OF ANYTHING NEW IS REPORTED, ONCE



Coarse on purpose

A whole network and a whole site, not an address and a host, so a service that moves between neighbouring addresses or a site with a new subdomain is not new each time. A short built-in list of 63 two-label suffixes (`co.uk` , `com.au`) keeps names right without the full public suffix list; for one it lacks, a name is learned too wide, which is quieter and never louder.

Only what the tap sees

Everything is learned from the VM's own tap, so every detection is guest-attributed. A host connect belongs to QEMU and is never learned. A VM first seen later has its own learning period, so a new VM does not alarm for its first day.

```
$ shukrctl baseline
BASELINE learning period 24h0m0s, at most 20 new-item alerts per VM per day
vm=web-01 learning until 2026-09-21T03:00:00Z (destination 4, dns-suffix 2, inbound-peer 0) new alerts today 0,
held back in all 0
vm=db-01 reporting (destination 9, dns-suffix 1, inbound-peer 1) new alerts today 3, held back in all 5
[medium] new-destination web-01 contacted 192.0.2.0/24 for the first time (tcp to 192.0.2.44:22)
as printed in docs/baselines.md (documentation example addresses)
```

Turned on by you

Off unless the rules file has a `baselines:` section. It needs `-data-dir:` without it, learning starts over with every restart and a daemon that restarts more often than the learning period would never report anything. `shukrctl doctor` warns about exactly that.

Forgetting is recorded

`shukrctl baseline VM --forget` (admin key only) restarts a VM's learning after a deliberate change. It could also hide a change, so it is recorded as a `baseline-forgotten` detection naming who did it.

A guest can hide inside its own learning period. Whatever it does in the first 24 hours becomes normal. Start a VM you do not trust under a watchlist (`destinations` , `dns`), not a baseline. Past the daily cap, new items are still learned and counted but not reported, and one `baseline-cap` detection says so.

BEYOND THE INCIDENT · NAMES

Which sites is this VM talking to?

A guest reaches a site by address, by a DNS lookup or by a TLS connection, and the last two say the name. Shukra reads that name from the first packet on the VM's own tap, so a guest that skips DNS, or asks a resolver over HTTPS, is still seen going somewhere.

DNS names

The first question of a plain query on UDP port 53, IPv4 or IPv6. The tap program copies at most 128 bytes and the daemon decodes the name and type. A repeat is announced once a minute. Not seen: answers, DNS over TCP, TLS or HTTPS, and a second question.

TLS server names

A ClientHello goes out before anything is encrypted and holds no application data. For a segment that starts one, the kernel copies up to 1,504 bytes; the daemon decodes it into one `guest_tls` event and throws the bytes away. It has its own budget of 100 events per tap per second.

guest_tls field	What it says
sni	the server name, lower-cased and printable. Empty when the hello has none, and never half a name
alpn, tls_version	the protocols offered (at most 8, such as <code>h2,http/1.1</code>) and the highest version the client offered
ja3	a fingerprint of the hello, present only when the whole hello was seen. It says what kind of client library made it, not who the client is
ech	the hello uses Encrypted Client Hello, so <code>sni</code> is at most the outer, public name and says nothing about the real site
tls_truncated	the hello did not end inside the copy

Where it stops

- **Cut short, and said so.** A hello longer than the copy, or split over segments (a hello with a post-quantum key share is about 1.5 KB), is read as far as it goes. The name is usually in the first bytes; there is no JA3, because a fingerprint of part of a hello would be a different fingerprint.
- **Hidden names.** With ECH only the outer name is visible. QUIC and HTTP/3 (UDP) are not read.
- **Only the hello,** not the server's answer: not whether the connection worked, or its certificate.

What you do with it

- `tls:` rules match a name by suffix, exact or contains, like `dns:` rules; the two do not judge each other's names.
- Learned baselines count a server name as the site it names, so a site looked up by DNS is not new when the VM connects to it.
- `shukractl watch` prints `sni=`, and the console lists the names, searchable by name, protocol and fingerprint.
- `-tls-events=false` makes the program read no TCP payload at all; `-dns-events=false` does the same for DNS.

TRACE											
Server names the guest asked for (TLS ClientHello, seen on the VM tap) (2)											
TS	VM	SNI	ALPN	TLS_VERSION	DST	DPORT	JA3	ECH	TLS_TRUNCATED	BLOCKED	
2026-09-19T09:42:08Z	redis-01	pool.nanopool.org	—	1.2	203.0.113.50	3333	—	—	yes	—	
2026-09-19T09:42:02Z	payment-prod-03	ledger.internal.example	h2,http/1.1	1.3	10.0.4.12	443	375c6162a492dfbf2795909110ce8424	—	—	—	

Showing the newest 2 of 2.

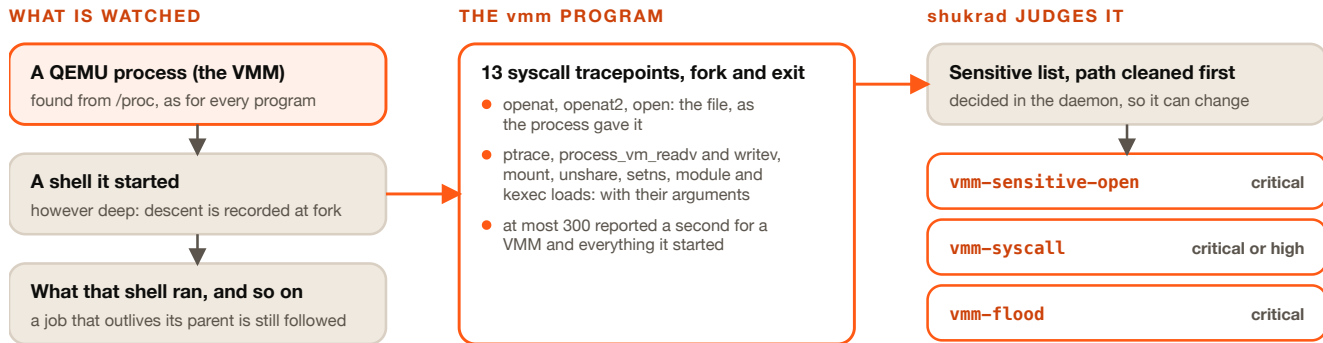
Illustrative: the Host connections page, rendered from fixture data. The first row is a hello cut short: no protocols, no fingerprint.

A name is sensitive. It says what a VM is doing, and a fingerprint what software it runs. Both are held in the event list, the flight recorder and an incident bundle, so treat them like the addresses beside them.

PROTECT · THE VMM ITSELF

Is the VMM doing something a VMM never does?

A QEMU process in steady state is quiet in a particular way: its disk images are already open and it makes none of the calls a normal program makes to reach around itself. So when one opens a file, or calls `ptrace`, `mount` or a module load, something is wrong: a guest that escaped into the VMM, or a person with a shell in it.



It reports; it does not stop anything. Isolation cuts the guest's network at its tap, not a process in the VMM's tree.

Nothing to configure

On the reference hypervisor, ten running QEMU processes made no `openat`, `openat2`, `open`, `ptrace`, `mount`, `unshare` or `setns` call in thirty seconds. So any one is worth a look, and the reports are very few.

What counts as sensitive

A built-in list: `/etc/shadow`, `sudoers`, `ssh keys`, `/proc/*/mem`, the docker and containerd sockets, `k3s` server files, and `/etc/shukra` and `/var/lib/shukra`, which hold Shukra's own key and data. A `vmm:` section adds `paths`, exempts some with `ignore` (a VM image under a watched directory) or narrows the calls.

Detection	When	Severity
vmm-sensitive-open	a watched process opens a path on the sensitive list	critical (configurable)
vmm-syscall	a watched process makes one of the ten calls	critical for <code>ptrace</code> , <code>process_vm_writev</code> , module loads and <code>kexec</code> ; high for <code>process_vm_readv</code> , <code>mount</code> , <code>unshare</code> , <code>setns</code>
vmm-flood	a VMM, with all it started, made more than 300 opens and calls in a second	critical

```

vmm-sensitive-open critical cat (pid 4321) in web's VMM process tree opened /etc/shadow, which matches /etc/shadow:
a VMM has no reason to
vmm-syscall critical gdb (pid 77) in web's VMM process tree called ptrace (request 16 (ATTACH) on pid 1): a
VMM has no business doing that
as printed in docs/vmm-tripwires.md. A VMM starting or stopping opens a few files: those are events, not detections
  
```

A tripwire, not a sandbox. The path is read when the call starts and a symlink is not followed. A VMM that does none of these is not seen. An open that fails still counts: a detection means a VMM process tried, not that it read the file.

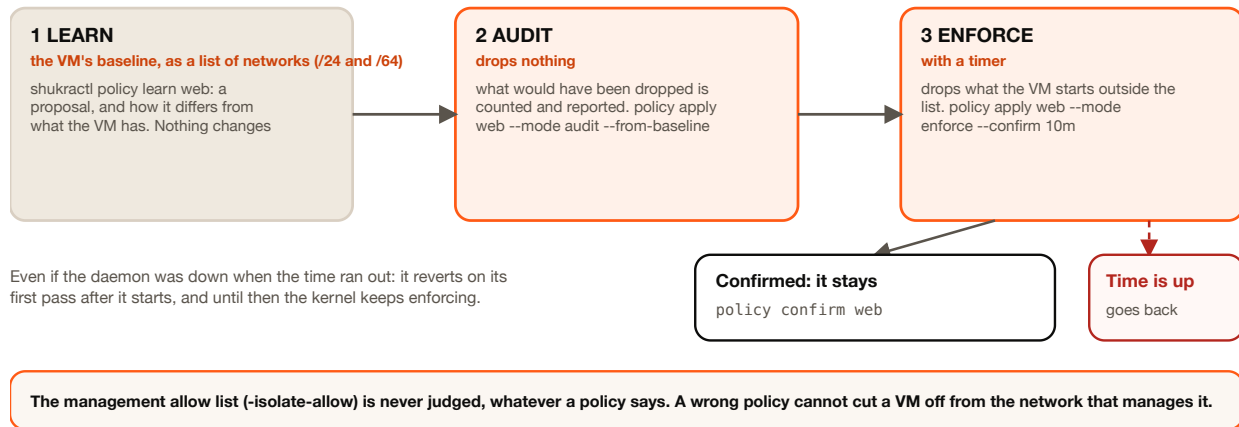
What it records, and costs. The path (up to 256 bytes) with the program name and pid, never contents. About 183 ns per open, 0.161% of one core on the reference host; `-vmm-tripwires=false` does not load it.

PROTECT · EGRESS POLICY

Where may this VM connect?

A VM's baseline says where it normally connects. An egress policy turns that into a list the tap program holds it to: the VM may start connections to these networks and no others. A wrong policy cuts a working VM off, so it is built to be tried before it is believed. Nothing changes for a VM until one is applied.

THREE STAGES, EACH SAFER THAN THE NEXT IS RISKY



What it judges

What the guest **starts**: a TCP SYN, or a UDP datagram that is not multicast and not an answer. A server keeps working under enforcement. ICMP, established connections, multicast and broadcast are not judged. Networks only, not ports or names.

What enforcing needs

The allow list (-isolate-allow), a -data-dir, a non-empty list, and --confirm (30 seconds to 24 hours) or --permanent. Audit needs none of them. Apply, confirm and remove need the admin key.

What stops a wrong policy doing harm

- The record is saved before the kernel is changed, and a change that fails is put back.
- The kernel keeps enforcing without the daemon through a crash, a restart or a stop. A tap enforcing a policy nobody recorded is an orphan that doctor flags.
- Every change is a detection (policy-applied, -confirmed, -reverted, -removed), so a webhook hears of it. A stray connection is a low egress-policy-audit or a medium egress-policy-blocked.

EGRESS POLICY				
2 VMs under a policy, 1 waiting to be confirmed				
Which networks a VM may start connections to. Audit reports what would be dropped and drops nothing. Enforcing goes back to what the VM had unless it is confirmed in time. A policy is proposed and applied with shukractl policy.				
VM	MODE	NETWORKS	SET BY	TAPS
payment-prod-03	enforce	4: 10.0.4.0/24, 10.0.0.0/24, 198.51.100.0/24 and 1 more	alice (baseline)	tap7: kernel enforce, judged 4210, 14 dropped
	unconfirmed: goes back to audit with 4 networks at 2026-09-20T03:10:00Z			Confirm Remove...
api-01	audit	1: 10.0.0.0/24	bob (manual)	tap1: kernel audit, judged 88, 7 would have been dropped
				Remove...

Illustrative: the Egress policy page, rendered from fixture data. One policy waits to be confirmed; the other is auditing.

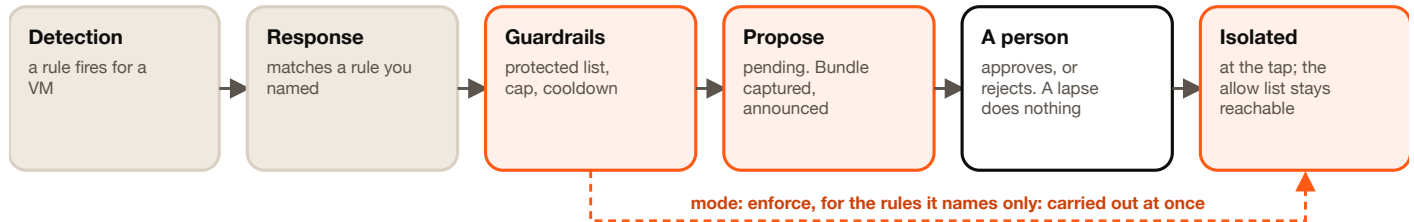
It is not a firewall. A resolver or time server the VM uses is a destination like any other: leave it off the list and lookups stop. A new tap has no policy for up to two seconds.

PROTECT · RESPONSES

Acting on a detection, safely

A detection tells a person something is wrong. A response can also do something about it, and the only thing it can do is isolate the VM. Because that cuts a running workload off, it is built so it cannot happen by accident. It is off until the rules file has a `responses:` section.

A RESPONSE CAN DO ONE THING: ISOLATE THE VM. BY DEFAULT IT ONLY PROPOSES



`dry_run`: true records what would have been done and changes nothing, so a response can be tried on real detections first.

Guardrail	What it does
never_isolate	no response may isolate a listed VM, not even an approved proposal
isolate must work	nothing is attempted without <code>-isolate-allow</code> and the tap program. A dry run is still recorded
max_per_hour, cooldown	at most 3 automatic isolations an hour by default (1 to 100); a VM is left alone for an hour after an action, so a storm is one action
expire, release_after	a proposal nobody decides lapses (30 minutes by default); a timed release survives a restart. Approving checks the guardrails again

The incident bundle

Every proposal carries the bundle `shukractl incident` gives: the verdict, the window's detections, up to 500 recorder events, the VM's isolate requests, the allow list and the programs, over the 15 minutes before the detection. Nothing from the daemon's configuration. Stored per action, mode 0600, as sensitive as the event list. `incident --at` gives the same for a past time.

What is recorded

Each decision is an action with an id, the detection behind it, who decided and what the isolate request answered. Each step is announced as a detection, so a webhook can page whoever approves. "Who" is a label the caller sends, `shukractl` or `api`: the API key is one key, not a login.

ACTIONS					
1 waiting for a decision					
ID	VM	RESPONSE	FOR	STATUS	WHAT HAPPENED
a-3	payment-prod-03	contain-miners (propose)	crypto-pool [high] payment-prod-03 looked up eth.nanopool.org (A)	pending	waiting, lapses 2026-09-20T03:30:00Z
a-2	batch-etl-01	contain-miners (propose)	crypto-pool [high] batch-etl-01 looked up nanopool.org (A)	executed	isolated: 1 tap (approved by alice (contain-miners, a-2)) releases itself at 2026-09-19T21:15:00Z
a-1	redis-01	contain-miners (propose)	crypto-pool [high] redis-01 looked up nanopool.org (A)	refused	redis-01 is protected: no response may isolate it

Illustrative: the Actions page, rendered from fixture data. One proposal waits; one was approved and releases itself; one was refused by `never_isolate`.

One action, and only at the tap. A response does not stop a VM, kill a process or change a firewall. A guest can cause a detection, so with an `enforce` response for a rule it can trigger it can get *itself* isolated, never another VM. Keep enforce rules narrow, and try one as `dry_run` first.

What Shukra reads, and what it never does

The accurate promise is stronger than "no agent": nothing runs inside a guest, and no application data is read, with three named exceptions. Each is bounded, decoded in the daemon, and has a switch.

By default

Metadata only. Counters and histograms from the host: exits, scheduling, block requests, TCP connects, tap packet counts. From a tap: headers, so flow addresses and ports, handshake outcomes and kernel drop reasons. VM names from the QEMU command line.

Three bounded exceptions

The name in a plain **DNS** query, the server name in a **TLS ClientHello**, and the **file names** a VMM opens. Answers, the rest of a TLS connection and file contents are not read. Host task names (`comm`) also appear.

Never

Anything inside a guest. Application payloads. The guest's own steal counter. Which process in the guest made a connection. Keys in any API response, in `doctor` or in `systemctl show`.

Exception to "no payloads"	How much is copied, and what is kept	Turn it off
DNS names	the first question of a plain query on UDP port 53: at most 128 bytes, kept as the name (cut at 253 bytes) and the type	<code>-dns-events=false</code>
TLS ClientHello	up to 1,504 bytes of the segment that starts a handshake, before anything is encrypted. The daemon keeps the server name, protocols, version and a JA3 fingerprint, and discards the bytes	<code>-tls-events=false</code> : no TCP payload is read at all
VMM file names	the path a QEMU process, or a program it started, opens (up to 256 bytes) with the program name and pid. Never the contents	<code>-vmm-tripwires=false</code>

All three are sensitive: a name says what a VM is doing, a fingerprint what software it runs, and a path can name a person or a secret. The scheduler program also records the command name of a host task that took a vCPU's CPU, so names such as `kubect1` can appear in the API.

Question	Answer
Does anything run inside the VM?	No. There is no agent. Everything is read from the host: <code>/proc</code> , kernel tracepoints, and the host side of each VM's tap.
Are host TCP events the guest's?	No. Host connects are the QEMU process's own sockets, labelled <code>qemu-process</code> and <code>guest_attributed: false</code> , as is what a VMM opens or calls. Only events seen on a VM's tap are <code>guest-tap</code> and <code>guest-attributed</code> , and only for a tap that belongs to a VM in the current scan. A tap no VM owns stays <code>unattributed</code> .
Can a guest change what is attributed to it?	No. Attribution comes from the tap's owner, found on the host, not from anything in a packet. A guest cannot make another VM's traffic look like its own, and a detection names the VM whose tap it was seen on.
Does the advisor change anything?	No. Right-size advice is text for a person, with the evidence and a confidence. Nothing resizes a VM.
What does an incident bundle expose?	VM names, addresses, DNS and TLS names and file paths, as the event list does. Nothing from the daemon's configuration except the allow list. It is written with mode 0600.

What this release will not pretend. It can say *which VM and which address*, not *which process inside the guest*. A VM with user-mode networking has no tap and so no guest traffic, and `shukract1 doctor` names it. Treat the data directory as sensitive, and alert sinks as a place that data leaves the host.

Who can do what to Shukra, and to your VMs

Shukra is a privileged daemon on a hypervisor, next to the VMs it watches. The parties that touch it are not equally trusted, and every table a guest can fill is bounded.

Party	Trusted?	What it can do, and what limits it
The host: kernel, root, <code>/proc</code>	entirely	Shukra believes <code>/proc</code> . Host root can stop, replace or read it: it is not a defence against a compromised hypervisor .
The operator, with the admin key	yes	Isolate or release any VM, apply, confirm or remove an egress policy, approve or reject a response, forget a baseline. One key, no per-person identity: the recorded actor is a label the caller supplies.
A holder of the read-only key	for reading	Reads everything, including <code>/metrics</code> , the stream, the recorder and incident bundles. Cannot change anything: every request that is not <code>GET</code> or <code>HEAD</code> is refused. Give this one to Prometheus.
A caller with no key	no	Gets <code>/healthz</code> , <code>/readyz</code> and the console's static files, and nothing else.
A guest	no	Chooses every byte it sends. It can make Shukra do work and record events, within fixed limits. It cannot change what is attributed to it, cannot read or write anything of Shukra's through the tap, and cannot get another VM isolated, dropped by a policy or blamed.
A VMM: QEMU and what it starts	watched, not trusted	It is where a guest escape lands, so it is what the tripwires watch. A taken-over VMM can rewrite its own command line, which is how Shukra names a VM's tap. A VMM that is root on the host is host root, above.
Where alerts go	not with your data	A webhook, syslog or file receives every detection, which names VMs and addresses. The webhook is signed when a secret is set.

What a guest can do is generate load

A very high packet rate costs CPU in the tap program on that tap, and a flood of unanswered SYNs makes the pending table forget its oldest entries. Neither hides the flood: the counters see every frame. Text from a guest is lower-cased, cut at 253 bytes and made printable before it goes anywhere.

Every table is bounded

Pending handshakes and UDP flows 65,536 each, tables of names already seen 16,384 each, a VMM's reported calls 300 a second, the event list 2,048 events in six classes with guaranteed shares, the flight recorder 4,096 per VM. At the bound the oldest entry is forgotten and the counters still count.

Privilege

The service runs as root with six capabilities, `NoNewPrivileges`, a read-only filesystem apart from its data directory, and a restricted set of address families. `CAP_SYS_PTRACE` and `CAP_DAC_READ_SEARCH` only read a libvirt VM's tap names from `/proc/<pid>/fd`: drop them if you run no libvirt VMs.

What enforcement does not cover

Isolation and a policy drop what crosses a VM's tap. They do not touch vhost-user or SR-IOV interfaces, a tap outside the host's network namespace, or anything that is not network traffic (vsock, virtio-serial). ARP and IPv6 neighbour discovery always pass under isolation.

The API key is the admin key. With the dev key, anyone who can reach the port can read everything Shukra knows and isolate any VM on the host. See the deploy page, and run `shukractl doctor` on every host.

The console: sixteen pages, five groups

The console is served by the daemon on port 30970 and talks only to its API. Every page says what it knows, how it knows it, and what it cannot see.

Overview
Programs attached,
VMs found,
detections.

Investigate
Virtual machines,
Flight recorder,
Explain.

Diagnostics
KVM, Scheduler,
Contention, Right-
size, Block,
Programs.

Network
Host connections
(with TLS names),
Drops.

Security
Detections,
Actions, Egress
policy, Isolate.

VM

At

Look back

payment-prod-03

dd/mm/yyyy, --:-- --

last minute

Download incident bundle

FINDINGS

Where to look, best supported first

Looking at: the last 1m0s

Noisy Neighbour HIGH CONFIDENCE

Another VM, api-01, took most of the CPU this VM's vCPUs were denied. Look at api-01's load, and at CPU pinning or separating the two onto different cores.

api-01 took 190 ms of the 240 ms the vCPUs were preempted (79%). shukractl trace contention shows what api-01 was doing meanwhile.

Storage Latency HIGH CONFIDENCE

Block requests from QEMU take long. Look at the backing device, its queue depth and other writers.

Block write p99 is up to 33.6 ms (slowest 28 ms) across 940 requests.

Cpu Preempted HIGH CONFIDENCE

The best CPU ever from this VM to CPU is. Look at what else is running on these best CPU for other VMs to best

Explain: ranked findings in the daemon's own words, an At field for a past time, and the incident bundle download.

TRACE

Packets the kernel dropped on VM taps

VM	TAP	KERNEL DROPS	SHUKRA DROPPED	OTHER DROPS	QUEST NOT READING
payment-prod-03	tap7	1290	0	1204	0
api-01	tap1	14	0	3	11
redis-01	tap2	0	0	0	0

Drops: what the kernel dropped on each tap, by reason and freeing function, and whether it was Shukra.

ENFORCEMENT

Isolate is enforced on the VM tap

An isolated VM keeps ARP, IPv6 neighbour discovery and traffic to or from 10.0.0.0/24, 10.0.9.4. Everything else on its tap is dropped.

VM

payment-prod-03

Isolate...

Release

Cut payment-prod-03 off from the network? Its tap will drop everything except ARP, IPv6 neighbour discovery and 10.0.0.0/24, 10.0.9.4. It stays isolated until you release it. It stays isolated if the daemon stops or crashes. Release it here, or with shukrad -detach-all if the daemon is down.

Isolate payment-prod-03

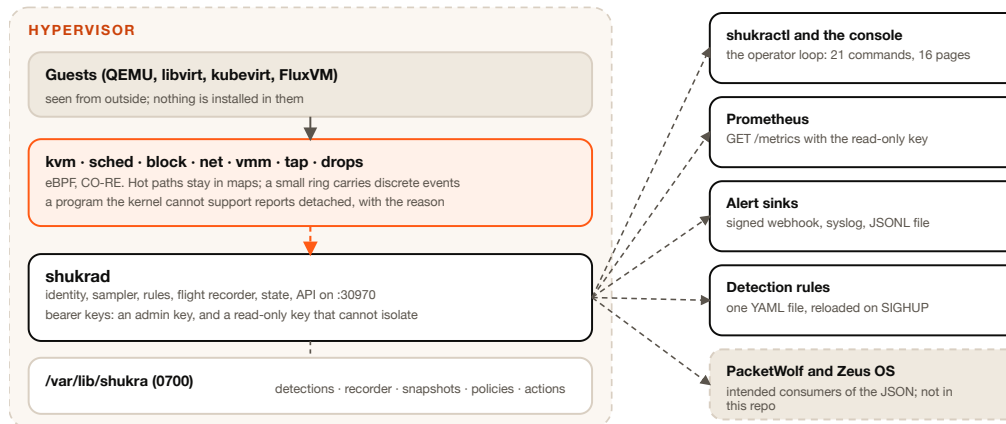
Cancel

Isolate: cut a VM off except the management network, with a confirmation step and what survives a daemon crash.

Illustrative: all captures are rendered from the console's fixture data. They are not a live host or customer data. Contention, Right-size, Host connections, Egress policy and Actions appear on earlier pages.

Rules, alerts, state and the API

Detection is one YAML file. Alerts leave through sinks you choose. State survives a restart with a data directory. Everything the console shows is a plain API call: 37 routes in all.



Detection rules

One file (`-watchlist`), re-read on `systemctl reload shukra` and checked offline with `shukractl rules check`. A bad file keeps the previous rules.

```

destinations: # out to, or in from
- {cidr: 185.0.0.0/8, name: egress, severity: high}
dns: # a name the guest looked up
- {name: pool, suffix: nanopool.org}
tls: # a server name in a ClientHello
- {name: tor, suffix: torproject.org}
thresholds: # a per-VM metric over a window
- {name: slow, metric: block_write_p99_ms, value: 50}
responses: # off unless present
- {name: contain, rules: [pool], action: isolate}

```

A detection keeps the attribution of the event that caused it. A rule says nothing until a full window of history exists. Or add a `baselines` section and let each VM teach it what is normal. `vmm` is optional: the tripwires need no section. `responses` defaults to propose.

Alert sinks

A **signed webhook** (HMAC, when a secret is set), **syslog** and a **JSONL file**.

State that survives a restart

File	Written
<code>detections.jsonl</code>	on every detection
<code>isolations.jsonl</code>	on every isolate request
<code>recorder.json</code>	every minute, and at shutdown
<code>actions.jsonl</code> , <code>incidents/</code>	each response decision, and its bundle
<code>policies.json</code>	on every policy change
<code>snapshots.jsonl</code>	every 5 minutes; read on demand
<code>baselines.json</code>	every minute, if it changed

Directory `0700`, files `0600`, each log rolls at 16 MiB.

Monitor the daemon

Endpoint	Auth	What
<code>/healthz</code> , <code>/readyz</code>	none	process is up; <code>200</code> after the first scan, <code>503</code> before
<code>/metrics</code>	bearer	Prometheus text. A VM with no measured counters has no series, not a zero
26 GET routes under <code>/api/v1</code>	bearer (read-only is enough)	status, vms, programs, seven traces, explain, incident, advice, baseline, policy, actions and more. <code>/stream</code> is server-sent events
8 POST routes	admin key	<code>isolate</code> , <code>release</code> ; policy <code>apply</code> , <code>confirm</code> , <code>remove</code> ; action <code>approve</code> , <code>reject</code> ; baseline/ <code>forget</code> . The read-only key gets <code>403</code>

Deploy, requirements, and how it is tested

Shukra is a systemd unit on the hypervisor, not a Helm chart, because eBPF has to run where the VMs run. One script installs it; a tarball and a `.deb` cover hosts without a compiler.

Install

```
$ ./scripts/deploy-remote.sh 10.0.1.5 sus
# build on the host, install, start
# shukra.service on :30970, run doctor
$ make dist
# tarball and .deb (amd64, arm64)
```

Keys and extra flags go in `/etc/shukra/env` (mode 0600). Rules are read from `/etc/shukra/detections.yaml` and state is kept in `/var/lib/shukra`.

Requirements

To get	You need
daemon, CLI, API	Go 1.27+ to build, or a release package
console	Node 22 to build
kvm, sched, block, net, vmm	kernel BTF and <code>CAP_BPF</code> (5.8+)
drops	Linux 5.17+
tap, egress policy, isolation	Linux 6.6+, <code>CAP_NET_ADMIN</code>

Before a host is reachable: the API key is the admin key. The shipped unit listens on `0.0.0.0:30970` over plain HTTP, so the key crosses the network in clear: pass `-tls-cert` and `-tls-key`, bind to a management address, or firewall the port. A daemon started with no `SHUKRA_API_KEY` uses the dev token `shukra`. A fresh install generates a random key, but an install over an older deploy keeps the key it finds, so an old host may still run the dev key. `shukractl doctor` fails it on a non-loopback address.

Isolation, an enforcing policy and a response need `-isolate-allow`. Without a management allow list each is refused, so a VM you cut off can still be reached by you.

Tested for real, not just unit-tested

EVERY PUSH AND PR

Go tests (545 test functions), the console's 56 tests and build, the programs loaded into the runner's kernel, the egress policy checked packet by packet, the installer.

REAL KERNEL, TAP RIG

Guest traffic, isolation, drops, every handshake outcome, DNS and TLS names, egress policy and VMM tripwires, in a network namespace with a real veth.

REAL KVM GUESTS

Two guests booted by fluxvm on a runner with `/dev/kvm`: events attributed, packet counts equal the kernel's, an audit policy marks exactly what is outside its list. A real QEMU, asked over QMP to open `/etc/shadow`, raises a critical `vmm-sensitive-open` (58 of 58 checks on the reference hypervisor).

Exact, not approximate. On GitHub's kernel, 17 outbound attempts came out as 5 accepted, 6 refused, 4 never answered and 2 blocked, with 1 retransmit, and attempts = accepted + refused + never answered + blocked held. Isolation and an enforcing policy are shown to survive `kill -9`, a restart and a graceful stop, and to be lifted by `-detach-all`.

Release

A `v*` tag builds the tarball and `.deb` for amd64 and arm64. There is **no tagged release yet**: the changelog lists what has merged to `main`.

Honest limits. The live-guest test fails, rather than skips, with no KVM. A SYN is "never answered" after 3 seconds; in a SYN flood the oldest pending entries are forgotten uncounted.

Where Shukra fits, and a buying checklist

Shukra is one Zyor product, open source, and it works alone. What it does not do is stated here so the choice is clear.

What Shukra is

Runtime intelligence and security for KVM from the hypervisor: what a host measures about its VMs, what became of a guest's connections, what a VMM opens, and ways to hold a VM to a policy or cut it off. Guests untouched.

What it is not

Not an agent in the guest, so it cannot name a process inside one. Not a firewall: isolation and an egress policy act on one VM's tap, and a policy judges networks a VM starts connections to, not ports. Not a CNI. Not a sandbox for the VMM: a tripwire reports.

FluxVM

Recognised: QEMU, Cloud Hypervisor, Firecracker and `fluxvm-hypervisor` guests are named from FluxVM's own record and traced on their host veth.

PacketWolf and Zeus OS

The intended consumers of Shukra's JSON. They are not in this repository, and nothing here depends on them.

Buying checklist

- 1 Run `shukractl doctor` on a candidate hypervisor: exposure, what is attached, what it cannot see, each finding with a fix.
- 2 Start observe-only. Read Overview, Programs and one real slow VM's Explain. Nothing is dropped.
- 3 Check the kernel: 5.17 for drops, 6.6 for the tap, egress policy and isolation. Read what each host reports as detached, and why.
- 4 Write a small rules file (add `baselines:` for first sightings), check it with `shukractl rules check`, and wire one alert sink. Give Prometheus the read-only key.
- 5 Set a real API key, and TLS or a management address, before any non-lab host is reachable. Check older deploys for the dev key.
- 6 Rehearse isolate on a test VM with an allow list: confirm it is refused without one, survives a daemon restart, and releases.
- 7 Try an egress policy on one VM: learn, audit for a day, then enforce with `--confirm` and let one run out to watch it revert. Turn a response on as `dry_run` or `propose`, read the bundle before approving, and list your primary databases in `never_isolate`.
- 8 Run a day with a data directory, then ask `explain --at` and `incident` about a past time. Read the resolution line. Read the "neither" figure of `shukractl advise` before trusting an over-provisioned verdict.
- 9 Decide whether PacketWolf or Zeus OS will consume the JSON, and which key they get.

See Explain Attribute Watch the VMM Contain Integrate Trust

every VM from the host · ranked causes with evidence · who took the CPU · what the VMM opens · a VM held to a policy or cut off · tested on real kernels and real KVM guests

See the VM. Explain the slowdown. Touch nothing inside the guest.

Talk to Zyor → zyvor.dev · source: github.com/zyvorai/shukra

© 2026 Zyor · Apache License 2.0 · eBPF runtime intelligence and security for KVM. Screenshots are rendered from the console's fixture data; diagrams are conceptual; measured figures are quoted from the project's own documentation and depend on your kernel, hardware and traffic.